

# Workshop 03 - Configuración de entorno Laravel con Apache, MariaDB, Composer y Node.js

---

## Portada

**Nombre del estudiante:** Ignacio Araya Rojas **Curso:** ISW811 - Software Libre **Nombre del taller:** Workshop 03 - Configuración de entorno de desarrollo Laravel **Fecha de entrega:** 19 de Junio del 2026 **Nombre del docente:** Misael Matamoros

---

### Repositorio GitHub

Repositorio utilizado para almacenar este Workshop03 y futuros workshops del curso:

<https://github.com/nachoar24/Workshops---Software-Libre>

Ruta del Workshop03 dentro del repositorio:

Workshop03/

---

## 1. Introducción

En este taller se configuró un entorno de desarrollo web utilizando una máquina virtual **Debian 12**, administrada mediante **Vagrant** y **VirtualBox**, desde una máquina anfitriona con sistema operativo Windows.

El objetivo principal fue preparar un ambiente funcional para ejecutar una aplicación **Laravel** utilizando **Apache** como servidor web, **MariaDB** como sistema gestor de base de datos, **Composer** como gestor de dependencias PHP y **Node.js/npm** para herramientas modernas de desarrollo frontend.

Durante el desarrollo del taller se creó un proyecto Laravel llamado:

```
larasite.local
```

Además, se configuró una base de datos llamada:

```
larabase
```

y un usuario de base de datos llamado:

```
larauser
```

Finalmente, el proyecto Laravel fue servido desde Apache mediante el dominio local:

```
http://larasite.local
```

Esto permitió comprobar que la aplicación funciona correctamente desde el navegador de la máquina anfitriona, sin depender únicamente del servidor de desarrollo `php artisan serve`.

## 2. Estructura del entregable

El entregable del taller se organizó dentro de la carpeta `Workshop03`, con la siguiente estructura:

```
Workshop03/  
├── README.md  
├── README.pdf  
├── images/  
│   ├── 01-vagrant-status.png  
│   ├── 02-php-apache-mariadb-version.png  
│   ├── 03-composer-version.png  
│   ├── 04-node-npm-version.png  
│   ├── 05-laravel-installer.png  
│   ├── 06-laravel-project.png  
│   ├── 07-env-config.png  
│   ├── 08-php-artisan-migrate.png  
│   ├── 09-apache-configtest-status.png  
│   ├── 10-hosts-larasite.png  
│   ├── 11-laravel-browser.png  
│   └── 12-dbeaver-larabase.png  
├── apache-conf/  
│   └── larasite.local.conf  
├── database/  
│   └── evidencia-larabase.md  
└── larasite.local/  
    └── código fuente del proyecto Laravel
```

La carpeta `images/` contiene las capturas utilizadas como evidencia del proceso. La carpeta `apache-conf/` contiene una copia del archivo de configuración del virtual host de Apache. La carpeta `database/` contiene evidencia adicional de la base de datos `larabase`. La carpeta `larasite.local/` contiene el código fuente del proyecto Laravel.

No se incluyeron carpetas innecesariamente pesadas como:

```
vendor/  
node_modules/  
.vagrant/
```

En su lugar, se conservaron los archivos necesarios para reconstruir el proyecto posteriormente, entre ellos:

```
composer.json  
composer.lock  
package.json
```

### 3. Inicio de la máquina virtual Debian 12

Primero se inició la máquina virtual desde Git Bash en Windows. Para ello, me ubiqué en la carpeta donde se encuentra el archivo **Vagrantfile**:

```
cd ~/ISW811/VMs/webserver
```

Luego se levantó la máquina virtual con:

```
vagrant up
```

Posteriormente, se ingresó a la máquina virtual mediante SSH:

```
vagrant ssh
```

Con estos comandos se verificó que la máquina virtual Debian 12 inició correctamente y que era posible acceder a ella desde la máquina anfitriona.

```
nacho@Mimica-Ignacio MINGW64 ~/ISW811/VMs/webserver (main)  
$ vagrant ssh  
Linux webserver 6.1.0-48-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.172-1 (2026-05-15) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Wed Jun 10 12:13:35 2026 from 10.0.2.2  
vagrant@webserver:~$ cd /vagrant/sites/larasite.local  
vagrant@webserver:/vagrant/sites/larasite.local$ cat /etc/apache2/sites-available/larasite.local.conf  
<VirtualHost *:80>  
    ServerName larasite.local  
    ServerAlias www.larasite.local  
  
    ServerAdmin webmaster@localhost  
    DocumentRoot /vagrant/sites/larasite.local/public  
    DirectoryIndex index.php index.html  
  
    <Directory /vagrant/sites/larasite.local/public>  
        Options FollowSymLinks  
        AllowOverride All  
        Require all granted  
    </Directory>  
  
    ErrorLog ${APACHE_LOG_DIR}/larasite_error.log  
    CustomLog ${APACHE_LOG_DIR}/larasite_access.log combined  
</VirtualHost>  
vagrant@webserver:/vagrant/sites/larasite.local$
```

## 4. Verificación del entorno LAMP

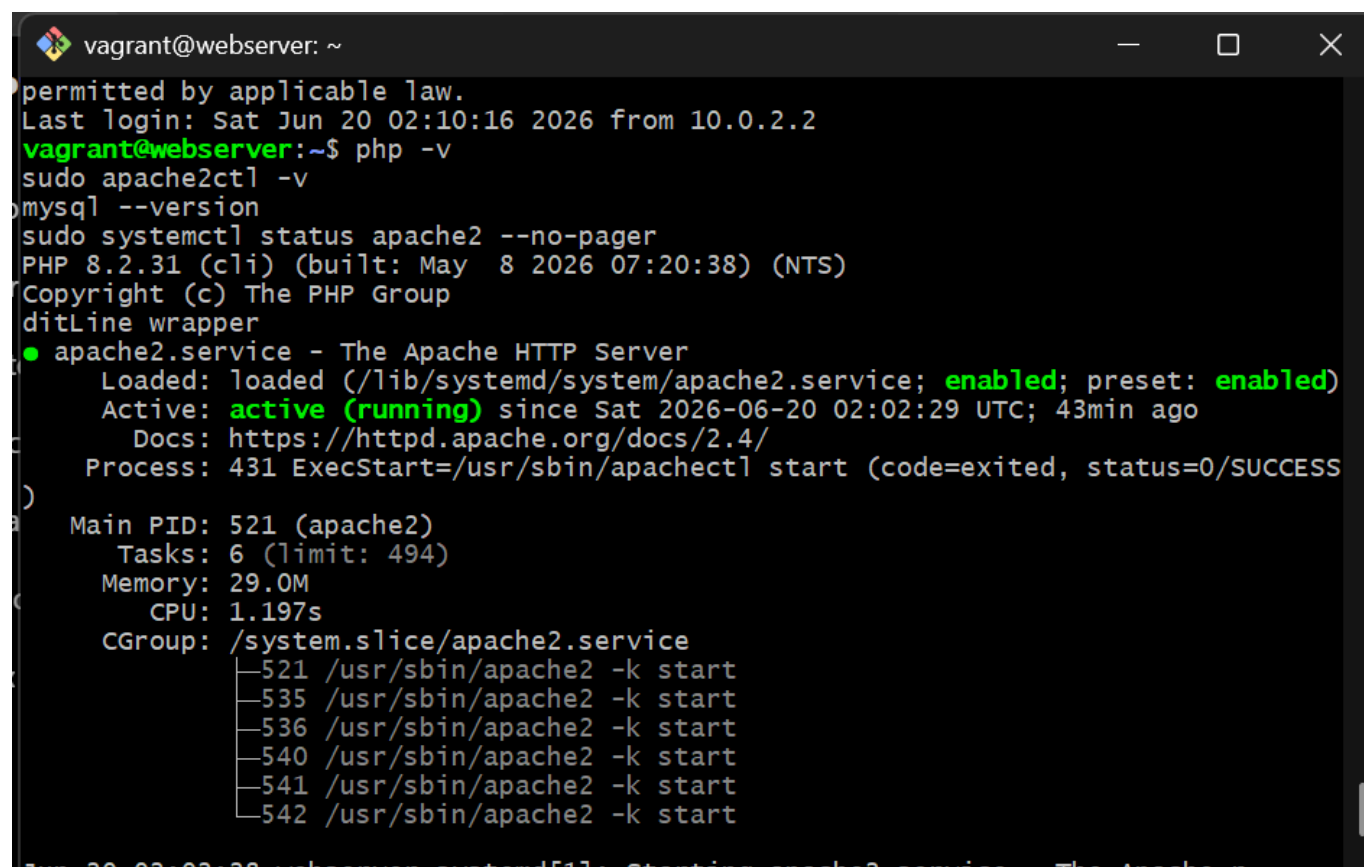
Una vez dentro de la máquina virtual, se verificó el funcionamiento de los principales componentes del entorno LAMP: PHP, Apache y MariaDB.

Los comandos ejecutados fueron:

```
php -v
sudo apache2ctl -v
mysql --version
sudo systemctl status apache2 --no-pager
```

Con estos comandos se confirmó que PHP estaba instalado, que Apache se encontraba disponible como servidor web, que MariaDB estaba instalado como sistema gestor de base de datos y que el servicio de Apache se encontraba activo.

Esta verificación fue importante porque Laravel requiere PHP para ejecutarse, Apache para servir la aplicación web y MariaDB para almacenar los datos del proyecto.



```

┌─┐ vagrant@webserver: ~
└─┘ permitted by applicable law.
Last login: Sat Jun 20 02:10:16 2026 from 10.0.2.2
vagrant@webserver:~$ php -v
sudo apache2ctl -v
mysql --version
sudo systemctl status apache2 --no-pager
PHP 8.2.31 (cli) (built: May 8 2026 07:20:38) (NTS)
Copyright (c) The PHP Group
ditLine wrapper
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-06-20 02:02:29 UTC; 43min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 431 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)

Main PID: 521 (apache2)
  Tasks: 6 (limit: 494)
 Memory: 29.0M
    CPU: 1.197s
   CGroup: /system.slice/apache2.service
           └─521 /usr/sbin/apache2 -k start
             └─535 /usr/sbin/apache2 -k start
               └─536 /usr/sbin/apache2 -k start
                 └─540 /usr/sbin/apache2 -k start
                   └─541 /usr/sbin/apache2 -k start
                     └─542 /usr/sbin/apache2 -k start
Jun 20 02:02:28 webserver systemd[1]: Starting apache2 service - The Apache HTTP Server...
```

## 5. Instalación y verificación de Composer

Composer se utilizó como gestor de dependencias para PHP. Esta herramienta es necesaria para instalar Laravel y administrar las librerías que el proyecto requiere.

Para verificar la instalación de Composer se ejecutó:

```
composer --version
```

El resultado mostró la versión instalada de Composer, confirmando que la herramienta quedó disponible globalmente dentro de la máquina virtual.

```
Jun 20 02:02:28 webserver systemd[1]: Starting apache2.service - The Apache...r...
Jun 20 02:02:29 webserver systemd[1]: Started apache2.service - The Apache ...ver.
Hint: Some lines were ellipsized, use -l to show in full.
vagrant@webserver:~$ composer --version
Composer version 2.10.1 2026-06-04 10:25:59
PHP version 8.2.31 (/usr/bin/php8.2)
Run the "diagnose" command to get more detailed diagnostics output.
vagrant@webserver:~$ |
```

## 6. Instalación y verificación de Node.js y npm

Laravel puede requerir herramientas del ecosistema JavaScript, especialmente para trabajar con Vite, recursos frontend y dependencias administradas mediante npm.

Node.js y npm fueron instalados mediante `nvm`. Para verificar que ambos quedaron correctamente disponibles, se ejecutaron los siguientes comandos:

```
node -v
npm -v
```

El resultado mostró las versiones instaladas de Node.js y npm.

```
Hint: Some lines were ellipsized, use -l to show in full.
vagrant@webserver:~$ composer --version
Composer version 2.10.1 2026-06-04 10:25:59
PHP version 8.2.31 (/usr/bin/php8.2)
Run the "diagnose" command to get more detailed diagnostics output.
vagrant@webserver:~$ source ~/.bashrc
vagrant@webserver:~$ node -v
v24.16.0
vagrant@webserver:~$ npm -v
11.13.0
vagrant@webserver:~$ |
```

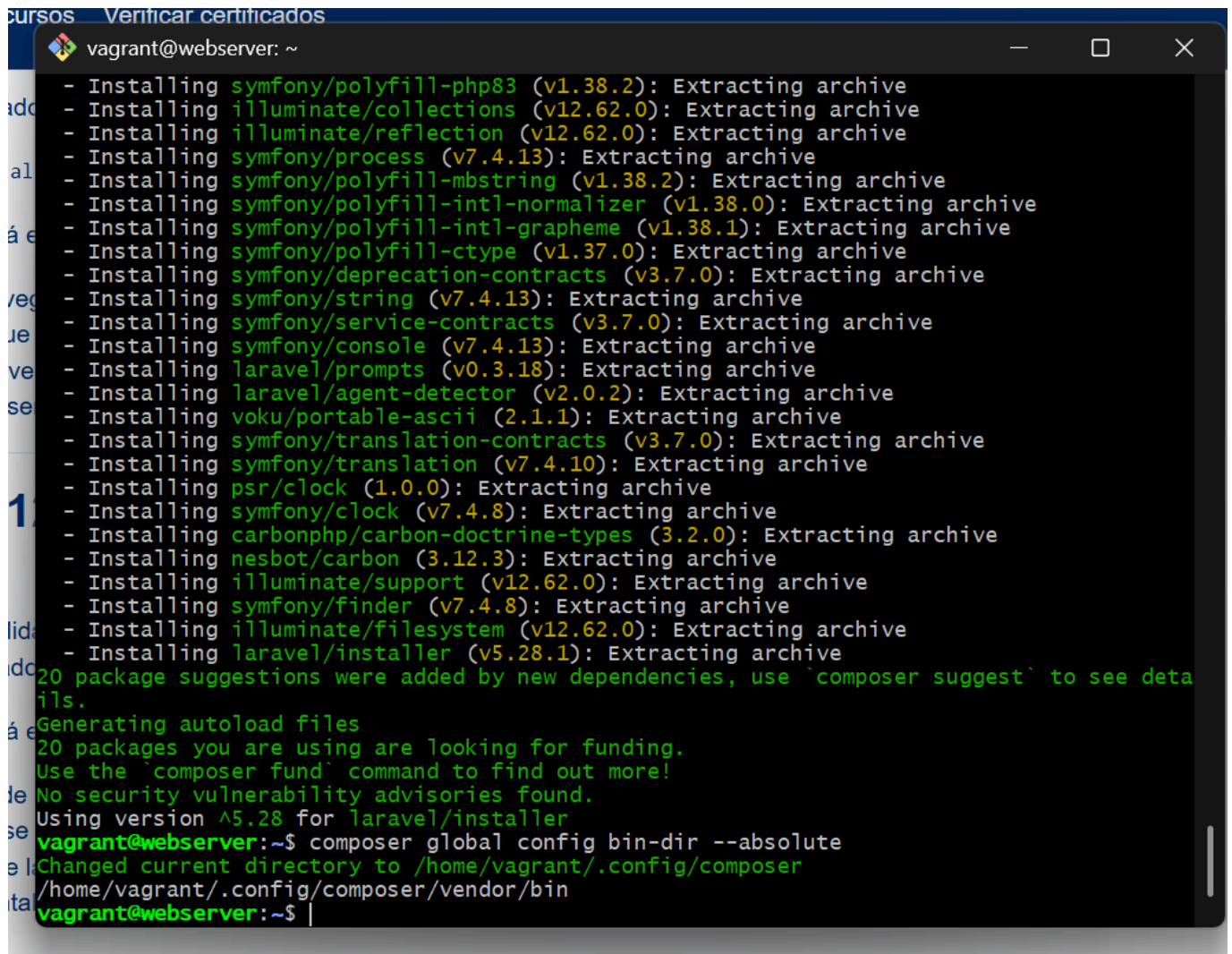
## 7. Instalación de Laravel Installer

Para facilitar la creación del proyecto Laravel, se instaló Laravel Installer mediante Composer.

La instalación se realizó con Composer y posteriormente se verificó con:

```
laravel --version
```

El resultado confirmó que Laravel Installer quedó disponible en la terminal de Debian y que era posible utilizarlo para crear proyectos Laravel.



```

cursos Verificar certificados
vagrant@webserver: ~
- Installing symfony/polyfill-php83 (v1.38.2): Extracting archive
- Installing illuminate/collections (v12.62.0): Extracting archive
- Installing illuminate/reflection (v12.62.0): Extracting archive
- Installing symfony/process (v7.4.13): Extracting archive
- Installing symfony/polyfill-mbstring (v1.38.2): Extracting archive
- Installing symfony/polyfill-intl-normalizer (v1.38.0): Extracting archive
- Installing symfony/polyfill-intl-grapheme (v1.38.1): Extracting archive
- Installing symfony/polyfill-ctype (v1.37.0): Extracting archive
- Installing symfony/deprecation-contracts (v3.7.0): Extracting archive
- Installing symfony/string (v7.4.13): Extracting archive
- Installing symfony/service-contracts (v3.7.0): Extracting archive
- Installing symfony/console (v7.4.13): Extracting archive
- Installing laravel/prompts (v0.3.18): Extracting archive
- Installing laravel/agent-detector (v2.0.2): Extracting archive
- Installing voku/portable-ascii (2.1.1): Extracting archive
- Installing symfony/translation-contracts (v3.7.0): Extracting archive
- Installing symfony/translation (v7.4.10): Extracting archive
- Installing psr/clock (1.0.0): Extracting archive
- Installing symfony/clock (v7.4.8): Extracting archive
- Installing carbonphp/carbon-doctrine-types (3.2.0): Extracting archive
- Installing nesbot/carbon (3.12.3): Extracting archive
- Installing illuminate/support (v12.62.0): Extracting archive
- Installing symfony/finder (v7.4.8): Extracting archive
- Installing illuminate/filesystem (v12.62.0): Extracting archive
- Installing laravel/installer (v5.28.1): Extracting archive
20 package suggestions were added by new dependencies, use `composer suggest` to see details.
Generating autoload files
20 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
Using version ^5.28 for laravel/installer
vagrant@webserver:~$ composer global config bin-dir --absolute
Changed current directory to /home/vagrant/.config/composer
/home/vagrant/.config/composer/vendor/bin
vagrant@webserver:~$ |
```

## 8. Creación del proyecto Laravel **larasite.local**

El proyecto Laravel fue creado dentro de la carpeta compartida entre Windows y Debian. En Debian, el proyecto se trabajó desde la ruta:

```
/home/vagrant/sites/larasite.local
```

También se puede acceder mediante la ruta:

```
/vagrant/sites/larasite.local
```

El proyecto se creó con el nombre:

```
larasite.local
```

Después de la creación del proyecto, se verificó su estructura y la versión de Laravel con los comandos:

```
cd /vagrant/sites/larasite.local  
ls -la  
php artisan --version
```

Durante esta parte se presentó un problema inicial: el proyecto fue creado, pero las dependencias de Composer no quedaron instaladas completamente. Al ejecutar `php artisan --version`, Laravel mostró un error indicando que no existía el archivo:

```
vendor/autoload.php
```

Esto indicaba que faltaba completar la instalación de dependencias. Para corregirlo, se ejecutó el siguiente comando dentro de la carpeta del proyecto:

```
composer install
```

Después de completar correctamente la instalación de dependencias, el comando `php artisan --version` funcionó correctamente.

```
vagrant@webserver: ~/sites/larasite.local

MariaDB [(none)]>
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| larabase |
| lospatitos |
| mysql |
| performance_schema |
| prueba |
| pruebaemoji |
| sys |
| workshop02 |
+-----+
9 rows in set (0.018 sec)

MariaDB [(none)]> EXIT;
Bye
vagrant@webserver:~/sites/larasite.local$
Message from syslogd@webserver at Jun 10 10:47:22 ...
kernel:[540371.280811] watchdog: BUG: soft lockup - CPU#0 stuck for 102s! [swapper/0:0]

Message from syslogd@webserver at Jun 10 11:05:44 ...
kernel:[541473.008375] watchdog: BUG: soft lockup - CPU#0 stuck for 799s! [swapper/0:0]

vagrant@webserver:~/sites/larasite.local$ ls -la .env
ls: cannot access '.env': No such file or directory
vagrant@webserver:~/sites/larasite.local$ cp .env.example .env
vagrant@webserver:~/sites/larasite.local$ nano .env
vagrant@webserver:~/sites/larasite.local$ php artisan key:generate

  INFO  Application key set successfully.

vagrant@webserver:~/sites/larasite.local$
```

## 9. Creación de la base de datos y usuario en MariaDB

Para conectar el proyecto Laravel con MariaDB, se creó una base de datos llamada:

```
larabase
```

También se creó un usuario llamado:

```
larauser
```

con la contraseña de laboratorio:

```
secret
```

Los comandos SQL utilizados fueron:

```
CREATE DATABASE larabase;

CREATE USER 'larauser'@'localhost'
IDENTIFIED BY 'secret';

GRANT ALL PRIVILEGES
ON larabase.*
TO 'larauser'@'localhost';

FLUSH PRIVILEGES;
```

Esta configuración permitió que Laravel se conectara a MariaDB utilizando un usuario específico para el proyecto, en lugar de utilizar el usuario administrador `root`.

---

## 10. Configuración del archivo `.env`

Laravel utiliza el archivo `.env` para definir variables de entorno, incluyendo la configuración de conexión a la base de datos.

En este taller se configuró el archivo `.env` para que Laravel utilizara MariaDB y la base de datos `larabase`.

La configuración aplicada fue:

```
DB_CONNECTION=mariadb
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=larabase
DB_USERNAME=larauser
DB_PASSWORD=secret
```

Para verificar la configuración se ejecutó:

```
cd /vagrant/sites/larasite.local
grep -n "^DB_" .env
```

Inicialmente, Laravel intentó utilizar SQLite porque el archivo `.env` todavía tenía configurado:

```
DB_CONNECTION=sqlite
```

Esto provocó que Laravel preguntara si se deseaba crear el archivo `database.sqlite`. Como el objetivo del taller era utilizar MariaDB, se canceló ese proceso y se corrigió el archivo `.env`.

Después de editar la configuración, se limpió la configuración cacheada de Laravel con:

```
php artisan config:clear  
rm -f bootstrap/cache/config.php
```

```
nacho@Mimica-Ignacio MINGW64 ~/ISW811/VMs/webserver (main)  
$ vagrant ssh  
Linux webserver 6.1.0-49-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.174-1 (2026-05-26)  
x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Sat Jun 20 02:45:25 2026 from 10.0.2.2  
vagrant@webserver:~$ cd /vagrant/sites/larasite.local  
vagrant@webserver:/vagrant/sites/larasite.local$ pwd  
/vagrant/sites/larasite.local  
vagrant@webserver:/vagrant/sites/larasite.local$ grep -n "^DB_" .env  
61:DB_CONNECTION=mariadb  
62:DB_HOST=127.0.0.1  
63:DB_PORT=3306  
64:DB_DATABASE=larabase  
65:DB_USERNAME=larouser  
66:DB_PASSWORD=secret  
vagrant@webserver:/vagrant/sites/larasite.local$ |
```

## 11. Ejecución de migraciones de Laravel

Después de configurar correctamente la conexión a MariaDB, se ejecutaron las migraciones de Laravel.

El comando utilizado fue:

```
php artisan migrate
```

Las migraciones permitieron crear automáticamente las tablas iniciales del proyecto Laravel dentro de la base de datos **larabase**.

Durante esta etapa también se presentó un error al intentar ejecutar antes el comando:

```
php artisan cache:clear
```

El error indicaba que la tabla **cache** todavía no existía. Esto ocurrió porque las migraciones aún no se habían ejecutado. La solución fue ejecutar primero:

```
php artisan migrate
```

Después de ejecutar las migraciones, Laravel creó las tablas necesarias en la base de datos.

```

vagrant@webserver: /vagrant/sites/larasite.local

vagrant@webserver:/vagrant/sites/larasite.local$ php artisan cache:clear

Illuminate\Database\QueryException

SQLSTATE[42S02]: Base table or view not found: 1146 Table 'larabase.cache' doesn't exist (Connection: mariadb,
Host: 127.0.0.1, Port: 3306, Database: larabase, SQL: delete from `cache`)

at vendor/laravel/framework/src/Illuminate/Database/Connection.php:838
834 | $exceptionType = $this->isUniqueConstraintError($e)
835 |     ? UniqueConstraintViolationException::class
836 |     : QueryException::class;
837 |
-> 838 | throw new $exceptionType(
839 |     $this->getNameWithReadWriteType(),
840 |     $query,
841 |     $this->prepareBindings($bindings),
842 |     $e,
    +23 vendor frames

24  artisan:16
    Illuminate\Foundation\Application::handleCommand()

vagrant@webserver:/vagrant/sites/larasite.local$ rm -f bootstrap/cache/config.php
vagrant@webserver:/vagrant/sites/larasite.local$ php artisan migrate

INFO  Preparing database.

Creating migration table ..... 111.92ms DONE

INFO  Running migrations.

0001_01_01_000000_create_users_table ..... 159.92ms DONE
0001_01_01_000001_create_cache_table ..... 78.80ms DONE
0001_01_01_000002_create_jobs_table ..... 117.23ms DONE

vagrant@webserver:/vagrant/sites/larasite.local$

```

## 12. Configuración del virtual host de Apache

Para servir el proyecto Laravel desde Apache, se creó el archivo de configuración:

```
/etc/apache2/sites-available/larasite.local.conf
```

El archivo contiene la configuración del virtual host para el dominio local `larasite.local`.

El contenido utilizado fue:

```

<VirtualHost *:80>
    ServerName larasite.local
    ServerAlias www.larasite.local

    ServerAdmin webmaster@localhost
    DocumentRoot /vagrant/sites/larasite.local/public
    DirectoryIndex index.php index.html

    <Directory /vagrant/sites/larasite.local/public>
        Options FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/larasite_error.log

```

```
CustomLog ${APACHE_LOG_DIR}/larasite_access.log combined  
</VirtualHost>
```

La directiva más importante en esta configuración es:

```
DocumentRoot /vagrant/sites/larasite.local/public
```

Laravel debe servirse desde la carpeta `public`, ya que esta contiene el archivo `index.php`, que funciona como punto de entrada de la aplicación. Por seguridad, Apache no debe apuntar directamente a la raíz del proyecto, sino únicamente a la carpeta `public`.

El sitio fue habilitado con:

```
sudo a2ensite larasite.local.conf
```

También se habilitó el módulo `rewrite`, necesario para que Laravel pueda manejar correctamente sus rutas:

```
sudo a2enmod rewrite
```

Después se validó la configuración de Apache:

```
sudo apache2ctl configtest
```

El resultado esperado fue:

```
Syntax OK
```

Finalmente, se reinició Apache y se verificó su estado:

```
sudo systemctl restart apache2  
sudo systemctl status apache2 --no-pager
```

Una copia del archivo `larasite.local.conf` fue incluida en el entregable dentro de:

```
apache-conf/larasite.local.conf
```

```
vagrant@webserver: /vagrant/sites/larasite.local

<Directory /vagrant/sites/larasite.local/public>
  Options FollowSymLinks
  AllowOverride All
  Require all granted
</Directory>

ErrorLog ${APACHE_LOG_DIR}/larasite_error.log
CustomLog ${APACHE_LOG_DIR}/larasite_access.log combined
</VirtualHost>
vagrant@webserver: /vagrant/sites/larasite.local$ sudo a2ensite larasite.local.conf
Site larasite.local already enabled
vagrant@webserver: /vagrant/sites/larasite.local$ sudo a2enmod rewrite
Module rewrite already enabled
vagrant@webserver: /vagrant/sites/larasite.local$ sudo apache2ctl configtest
Syntax OK
vagrant@webserver: /vagrant/sites/larasite.local$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-06-10 12:24:28 UTC; 22min ago
     Docs: https://httpd.apache.org/docs/2.4/
  Process: 64815 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Main PID: 64821 (apache2)
       Tasks: 7 (limit: 495)
      Memory: 36.0M
         CPU: 2.337s
    CGroup: /system.slice/apache2.service
            └─64821 /usr/sbin/apache2 -k start
              └─64822 /usr/sbin/apache2 -k start
                └─64823 /usr/sbin/apache2 -k start
                  └─64824 /usr/sbin/apache2 -k start
                    └─64825 /usr/sbin/apache2 -k start
                      └─64826 /usr/sbin/apache2 -k start
                        └─64836 /usr/sbin/apache2 -k start

Jun 10 12:24:28 webserver systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jun 10 12:24:28 webserver systemd[1]: Started apache2.service - The Apache HTTP Server.
vagrant@webserver: /vagrant/sites/larasite.local$
```

## 13. Configuración del archivo hosts en Windows

Para acceder al dominio `larasite.local` desde el navegador de Windows, fue necesario editar el archivo `hosts` de la máquina anfitriona.

En Windows, este archivo se encuentra normalmente en:

```
C:\Windows\System32\drivers\etc\hosts
```

La línea agregada fue:

```
192.168.56.10 larasite.local www.larasite.local
```

El archivo `hosts` permite asociar manualmente un nombre de dominio con una dirección IP. En este caso, se utilizó para que Windows resolviera `larasite.local` hacia la dirección IP privada de la máquina virtual Debian.

Al inicio, el comando:

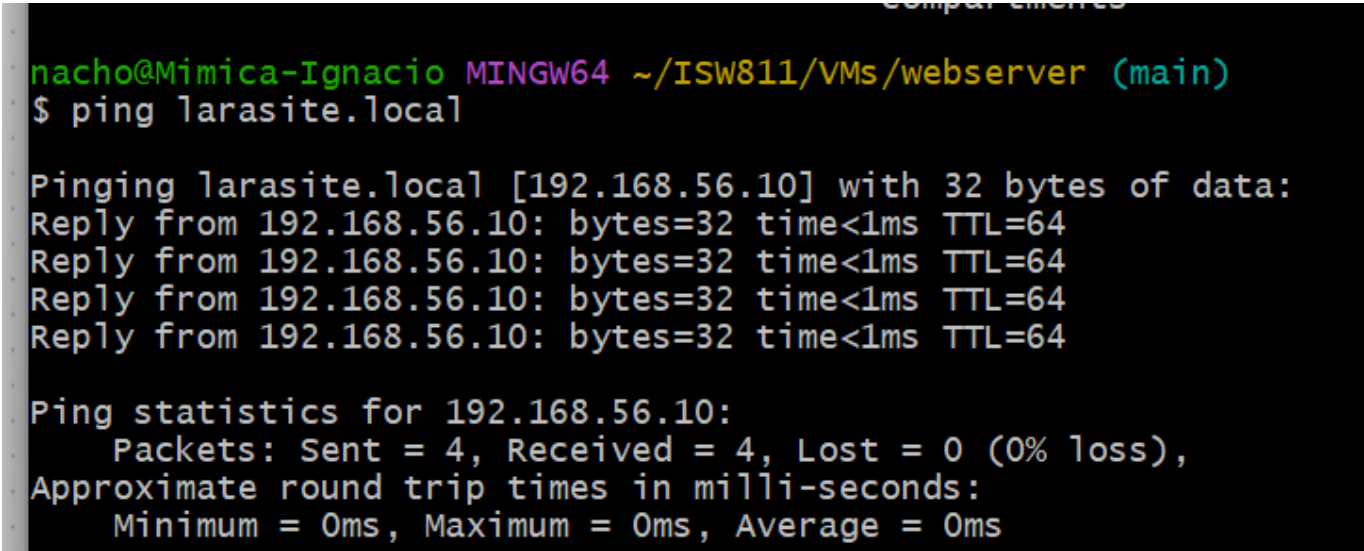
```
ping larasite.local
```

no resolvía el dominio. El problema se corrigió editando correctamente el archivo `hosts` como administrador y limpiando la caché DNS de Windows con:

```
ipconfig /flushdns
```

Después de este ajuste, el dominio resolvió correctamente hacia:

```
192.168.56.10
```



```
nacho@Mimica-Ignacio MINGW64 ~/ISW811/VMs/webserver (main)
$ ping larasite.local

Pinging larasite.local [192.168.56.10] with 32 bytes of data:
Reply from 192.168.56.10: bytes=32 time<1ms TTL=64
Reply from 192.168.56.10: bytes=32 time<1ms TTL=64
Reply from 192.168.56.10: bytes=32 time<1ms TTL=64
Reply from 192.168.56.10: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.56.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

---

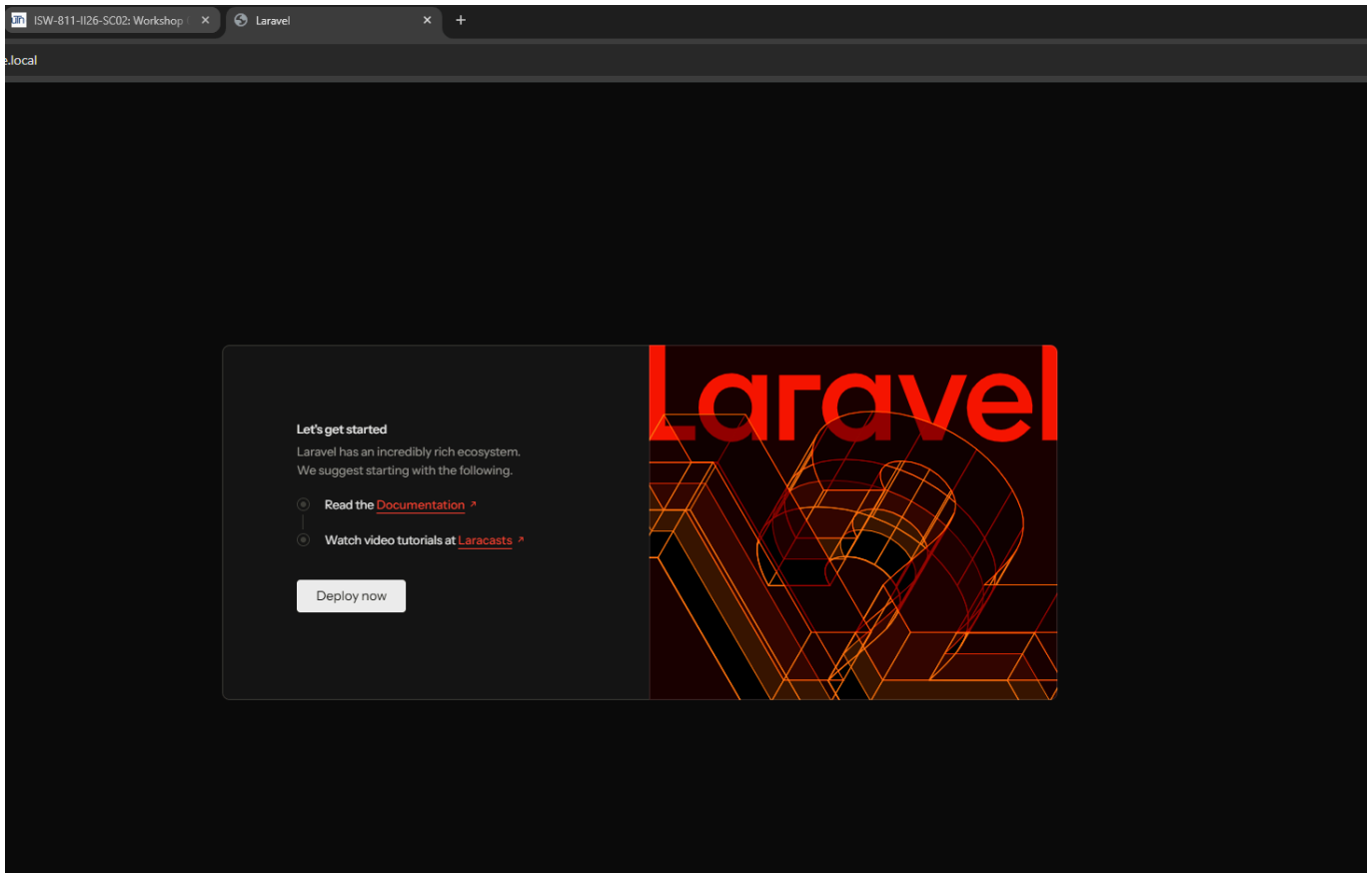
## 14. Verificación de Laravel desde el navegador

Después de configurar Apache y el archivo `hosts`, se abrió el navegador web de la máquina anfitriona y se ingresó a:

```
http://larasite.local
```

El resultado fue exitoso, ya que se mostró la página inicial de Laravel utilizando el dominio local configurado.

Esto confirma que el proyecto Laravel se está ejecutando mediante Apache y no únicamente mediante el servidor de desarrollo `php artisan serve`.



## 15. Evidencia adicional de base de datos con DBeaver

Como evidencia adicional, se utilizó DBeaver desde Windows para conectarse a la base de datos MariaDB dentro de la máquina virtual.

Para permitir la conexión desde Windows, se modificó la configuración de MariaDB. Primero se verificó el valor de `bind-address`:

```
sudo grep -n "bind-address" /etc/mysql/mariadb.conf.d/50-server.cnf
```

El valor inicial era:

```
bind-address = 127.0.0.1
```

Este valor solo permitía conexiones locales dentro de Debian. Para permitir la conexión desde la máquina anfitriona, se cambió a:

```
bind-address = 0.0.0.0
```

El comando utilizado fue:

```
sudo sed -i 's/^[:space:]*bind-address[:space:]*=.*\/bind-address = 0.0.0.0\/'
/etc/mysql/mariadb.conf.d/50-server.cnf
```

Luego se reinició MariaDB:

```
sudo systemctl restart mariadb
```

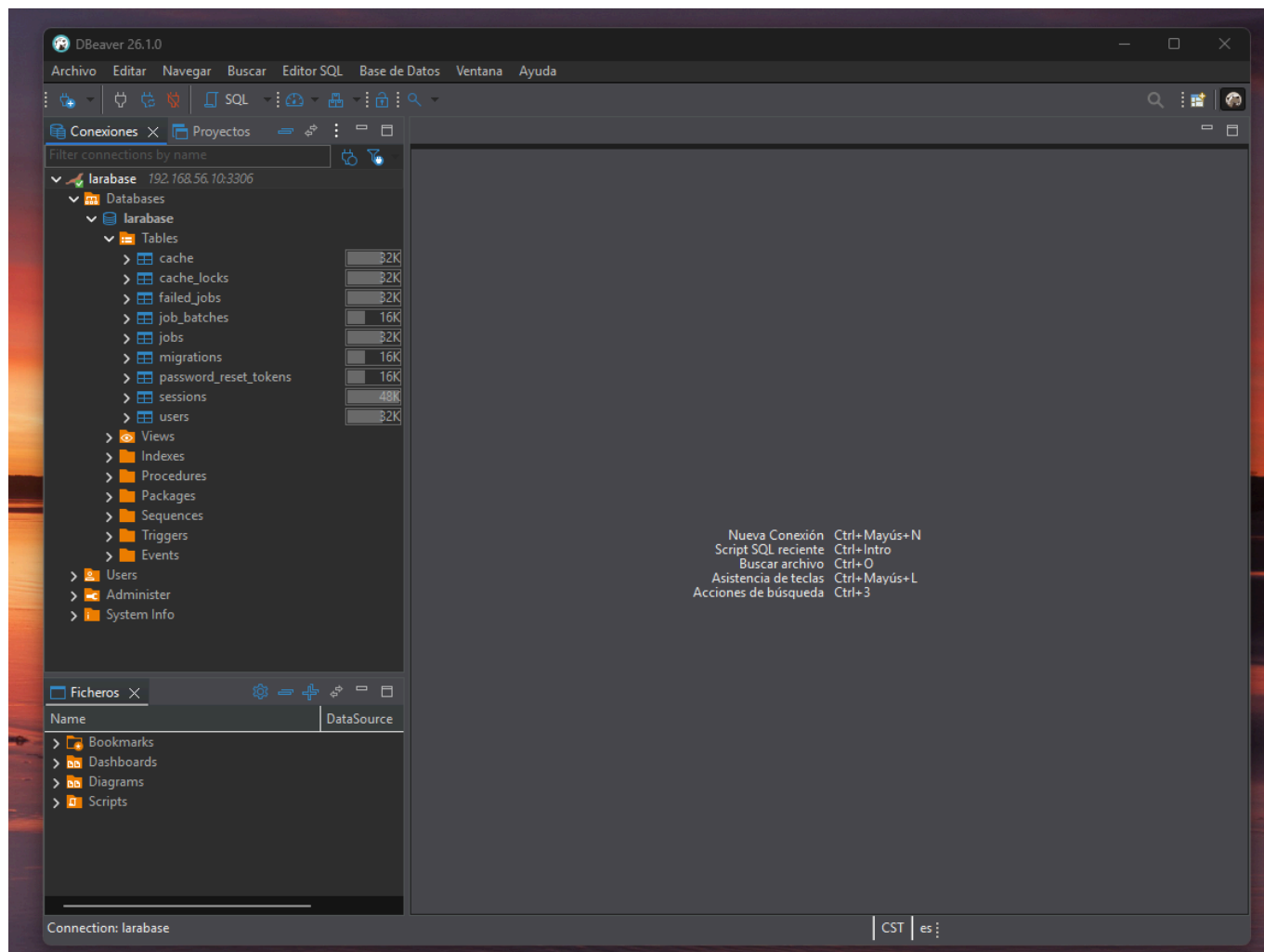
También se otorgaron permisos al usuario `larauser` para conectarse desde la red privada de VirtualBox:

```
sudo mysql -e "CREATE USER IF NOT EXISTS 'larauser'@'192.168.56.%' IDENTIFIED BY
'secret'; GRANT ALL PRIVILEGES ON larabase.* TO 'larauser'@'192.168.56.%; FLUSH
PRIVILEGES;"
```

La conexión en DBeaver se configuró con los siguientes datos:

```
Tipo de conexión: MariaDB/MySQL
Host: 192.168.56.10
Puerto: 3306
Base de datos: larabase
Usuario: larauser
Contraseña: secret
```

Después de realizar la conexión, se verificó que la base de datos `larabase` contenía las tablas generadas por Laravel.



## 16. Evidencia de tablas creadas en MariaDB

Además de la evidencia gráfica en DBeaver, se generó un archivo de evidencia en Markdown con el resultado del comando:

```
sudo mysql -e "USE larabase; SHOW TABLES;"
```

Este archivo se encuentra en:

```
database/evidencia-larabase.md
```

El objetivo de esta evidencia es demostrar que las tablas fueron creadas correctamente dentro de la base de datos **larabase** después de ejecutar las migraciones de Laravel.

## 17. Problemas encontrados y soluciones aplicadas

Durante el desarrollo del taller se presentaron algunos problemas que fueron corregidos durante la práctica.

### 17.1 Problema de resolución DNS en Debian

Al intentar descargar Composer, Debian inicialmente no podía resolver nombres de dominio. El mensaje indicaba un problema de resolución DNS.

Se verificó que la máquina virtual sí tenía conexión a internet mediante IP, pero no podía resolver dominios. La solución aplicada fue editar el archivo:

```
/etc/resolv.conf
```

y configurar servidores DNS públicos:

```
nameserver 8.8.8.8  
nameserver 1.1.1.1
```

Después de este ajuste, Debian pudo resolver dominios correctamente y se continuó con la instalación de Composer.

## 17.2 Instalación incompleta de dependencias de Composer

Después de crear el proyecto Laravel, el comando:

```
php artisan --version
```

falló porque no existía el archivo:

```
vendor/autoload.php
```

Esto indicaba que las dependencias no se habían instalado completamente. La solución fue entrar a la carpeta del proyecto y ejecutar:

```
composer install
```

Durante un primer intento, Composer tardó demasiado en descomprimir algunos paquetes. Posteriormente se repitió el proceso con la computadora fuera del modo de ahorro de batería y la instalación finalizó correctamente.

## 17.3 Laravel intentaba usar SQLite

Al ejecutar inicialmente:

```
php artisan migrate
```

Laravel mostró una advertencia relacionada con SQLite. Esto ocurrió porque el archivo `.env` todavía tenía configurada la conexión SQLite.

La solución fue corregir las variables de base de datos en `.env` para usar MariaDB:

```
DB_CONNECTION=mariadb
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=larabase
DB_USERNAME=larouser
DB_PASSWORD=secret
```

Después se limpió la configuración cacheada de Laravel con:

```
php artisan config:clear
rm -f bootstrap/cache/config.php
```

## 17.4 Tabla `cache` inexistente antes de migrar

Al ejecutar:

```
php artisan cache:clear
```

antes de las migraciones, Laravel intentó limpiar la tabla `cache`, pero esta todavía no existía en la base de datos.

La solución fue ejecutar primero:

```
php artisan migrate
```

Con esto se crearon las tablas necesarias, incluyendo la tabla `cache`.

## 17.5 Dominio local no resolvía en Windows

Después de configurar Apache, el navegador no cargaba `larasite.local` porque Windows todavía no resolvía el dominio. El comando:

```
ping larasite.local
```

indicaba que no se encontraba el host.

La solución fue editar el archivo `hosts` como administrador y agregar la línea:

```
192.168.56.10 larasite.local www.larasite.local
```

Luego se limpió la caché DNS con:

```
ipconfig /flushdns
```

Después de esto, el dominio resolvió correctamente y la página de Laravel cargó en el navegador.

---

## 18. Reconstrucción del proyecto

El entregable no incluye la carpeta `vendor/` ni la carpeta `node_modules/`, ya que son carpetas grandes que pueden reconstruirse a partir de los archivos del proyecto.

Para reconstruir las dependencias PHP del proyecto Laravel, se puede ejecutar:

```
cd larasite.local  
composer install
```

Para instalar dependencias de Node.js, se puede ejecutar:

```
npm install
```

En este proyecto no se incluye `package-lock.json`, ya que no fue necesario ejecutar `npm install` durante la creación del proyecto. Sin embargo, sí se conserva el archivo `package.json`.

---

## 19. Repositorio Git

El trabajo se mantiene dentro del repositorio Git de los talleres del curso.

**Repositorio GitHub:** <https://github.com/nachoar24/Workshops---Software-Libre>

**Ruta del taller dentro del repositorio:**

```
Workshop03/
```

Para registrar y subir los cambios del Workshop 03 se utilizaron los siguientes comandos:

```
git status  
git add Workshop03
```

```
git commit -m "Agregar Workshop03 Laravel"  
git push origin main
```

Esto permite evidenciar que el trabajo realizado fue versionado y almacenado en GitHub.

---

## 20. Conclusión

El Workshop 03 permitió configurar un entorno completo para desarrollo Laravel utilizando Debian 12, Vagrant, VirtualBox, Apache, MariaDB, Composer, Node.js y npm.

Como resultado final, se logró crear y ejecutar correctamente el proyecto:

```
larasite.local
```

También se configuró la base de datos:

```
larabase
```

se ejecutaron las migraciones de Laravel, se configuró Apache mediante un virtual host y se accedió correctamente al sitio desde la máquina anfitriona utilizando el dominio local:

```
http://larasite.local
```

El resultado final demuestra que Laravel fue servido correctamente desde Apache utilizando la carpeta **public** como punto de entrada, cumpliendo con los requisitos técnicos solicitados para el taller.